

Programming Assignment 3

DA6400: Jan-May 2026

1 Team Details

Kevin Kinsey S
EP23B027

Madhav Tadepalli
EE23B040

Soumya Lodha
EE23B140

1.1 Work Split

The work split was even across all tasks, with each team member running different hyperparameter configurations till we found the optimal configuration. After that, we ran all other questions together parallelly to increase time efficiency.

1.2 Links

Drive: -

Video: -

GitHub: <https://github.com/Maddy-1111/Reinforcement-Learning/tree/main>

2. Soft-Actor Critic

2.1. Pendulum

Q1 Reward Function Design

The reward is the cosine of the angular error, optionally penalised by angular velocity:

$$r_t = \cos(\theta_t - \theta_{\text{target}}) - 0.1 \cdot \tilde{\theta}_t^2$$

This is smooth, bounded in $[-1, 1]$, well-suited for learning stable target alignment behavior. It encourages the agent to reach the desired angle (via cosine term). The velocity term discourages oscillation near the target.

Q2 Code

See the submitted training script. SAC with auto- α is trained for 50 K env steps per target angle.

Q3 Learning Curves

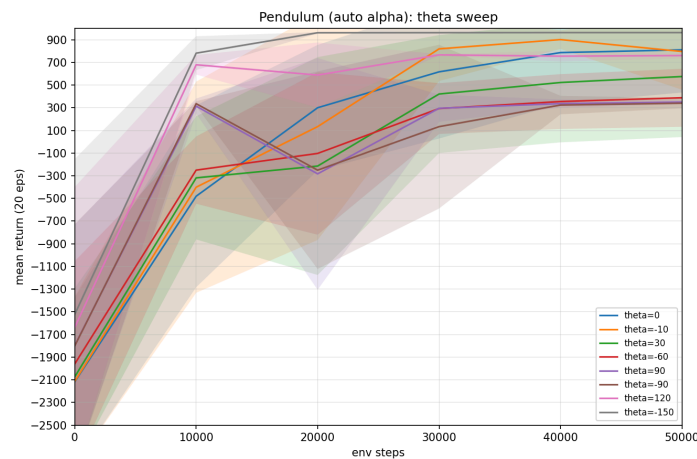


Figure 1: Learning curves for all target angles (mean $\pm$ 95% CI).

Learning speed:

- Targets like ($\theta = -150^\circ, 10^\circ, 120^\circ, 0^\circ$) learn the fastest. They show rapid improvement and reach high returns early ($\approx 40K$ steps).
- **Moderate angles** such as 30° and -60° learn more gradually, with steady but slower improvement.
- **Harder targets** like 90° and -90° initially struggle (sharp drop around 20K steps), indicating exploration difficulty before recovering.

Final performance (quality of behavior):

- **Best performance** is achieved by ($\theta = -150^\circ$), which consistently attains the highest returns ($950+$), indicating very stable alignment.
- ($\theta = -10^\circ$) and (0°) also achieve high returns (800), showing good convergence to the target.
- (120°) reaches moderately high performance (750) but with less improvement over time.
- (30°) achieves moderate returns ($550-600$), indicating partial success.
- **Lowest performance** is seen for (-60°), (90°), and (-90°) ($300-400$), suggesting difficulty in stabilizing at these angles.

Overall interpretation: Learning is faster and more stable for targets closer to the natural upright/downward equilibrium or those easier to reach via pendulum dynamics.

Targets at $\pm 90^\circ$ are hardest because they correspond to dynamically unstable points $\rightarrow$ gravity induces maximum deviation, requiring continuous precise torque, leading to sparse exploration and noisy learning signals.

Q4 Optimal Behavior

Two phases:

1. **Swing-up:** The agent injects energy via oscillatory motion to build momentum toward the target.
2. **Stabilisation:** Near equilibrium angles ($0^\circ, -10^\circ, -150^\circ$), the agent holds with slight deviation. For hard angles, it settles into a limit cycle or a shifted nearby angle (e.g. $\approx 140^\circ$ for a 120° target) since maintaining exact angle requires continuous torque.

This is reflected in the following visualizations of the system for θ target = -10° and 90° - [Google Drive](#)

Q5 Manual vs Automated Temperature α

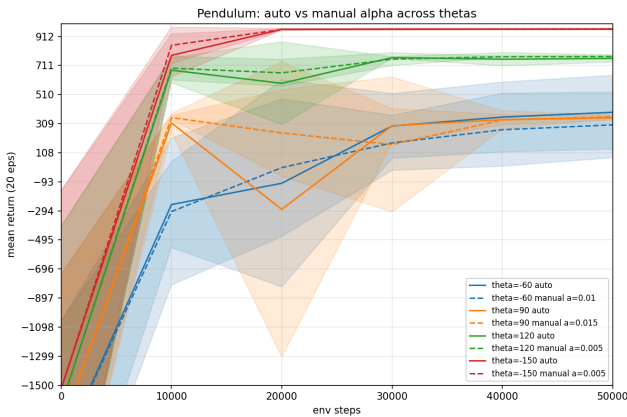


Figure 2: (a) Manual vs auto α .

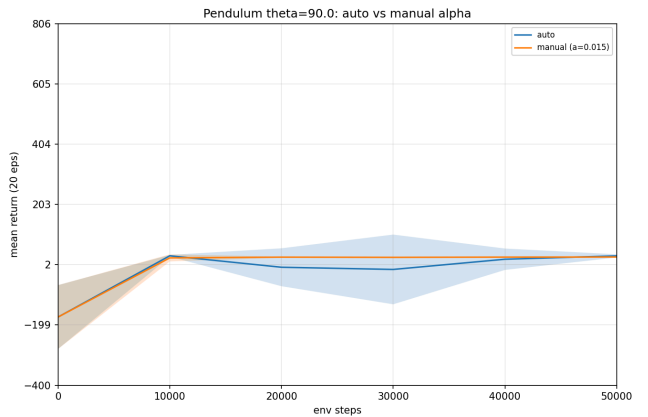


Figure 3: (b) Reward scaling $0.1 \times$.

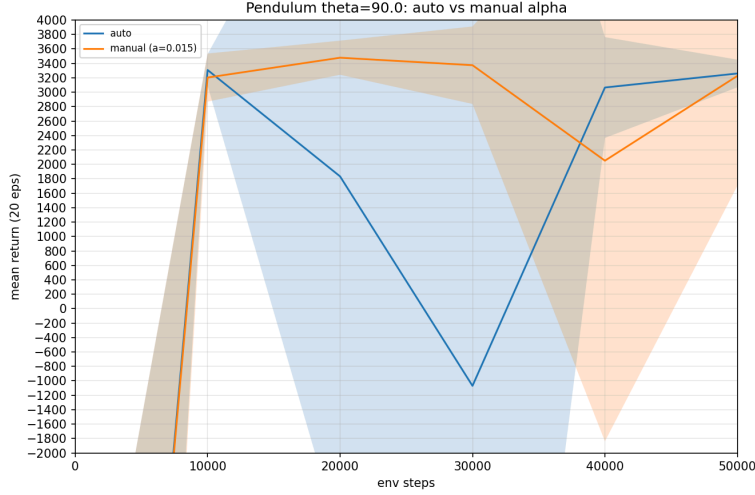


Figure 4: (b) Reward scaling $10\times$.

a) Manual vs Auto

We swept manual α values for each $\vartheta_{\text{target}} \in \{-60, 90, 120, -150\}$. The candidate values were chosen by inspecting the final 5–6 values of the auto-tuned α from the corresponding Q2.1.2 runs (the values α settles to in the last 10–20K env steps), and then refining around them. The final α_{mnl} values for every angle are:

θ_{target}	α_{mnl}
-60°	0.01
90°	0.015
120°	0.005
-150°	0.005

For $\theta \in \{90^\circ, 120^\circ, -150^\circ\}$, manual α achieves comparable or higher final return with tighter CI. The exception is $\theta = -60^\circ$: auto- α slightly outperforms because the optimal entropy shifts during training $\rightarrow$ a fixed scalar cannot adapt.

b) Reward Scaling

For $c = 0.1$, both methods converge to the same asymptote ($\approx +30 \equiv +300$ unscaled); manual is slightly tighter. For $c = 10$, manual converges cleanly ($\approx +3200$ by 10 K steps). Auto- α breaks down and returns crash to ≈ -1100 around step 30 K before partially recovering, with large variance.

$$\text{SAC objective } J = \mathbb{E}[\Sigma (r - \alpha \cdot \log \pi(a|s))]$$

Fixed (manual) α : the entropy term is unchanged, but the reward signal grows/shrinks by c . For $c = 10$, the reward dominates a fixed $\alpha = 0.015$, so the policy quickly becomes near-deterministic on a good action: same behaviour as the $c = 1$ case but reached faster. For $c = 0.1$, the entropy term is now relatively larger, so the policy stays a bit more exploratory but still converges.

Automated α : the dual update on α is driven by the current entropy of the policy relative to the target entropy. The actor and critic gradients, however, scale linearly with c . So with $c = 10$ the critic Q-values and actor losses are $10\times$ larger, but the α -loss is unchanged in magnitude: the optimiser pushes the policy toward determinism faster than α can compensate, the entropy collapses, learning becomes unstable, α has to ramp up by $10\times$ to catch up, and we get the large

mid-training dip and high variance that the plot shows. For $c = 0.1$ the opposite happens but it's gentler, because the policy is already near-deterministic and a slightly larger relative entropy term doesn't hurt.

Manual α adapts well to both scales because α_{mnl} was already in a robust regime. Auto α adapts well to $c = 0.1$ but is fragile to $c = 10$ - auto- α is not invariant to reward scale, and is one motivation for using a fixed manually-tuned α when the reward magnitude is known and stable.

2.2. Lunar Lander

Q1 Continuous SAC with Automated Temperature Tuning

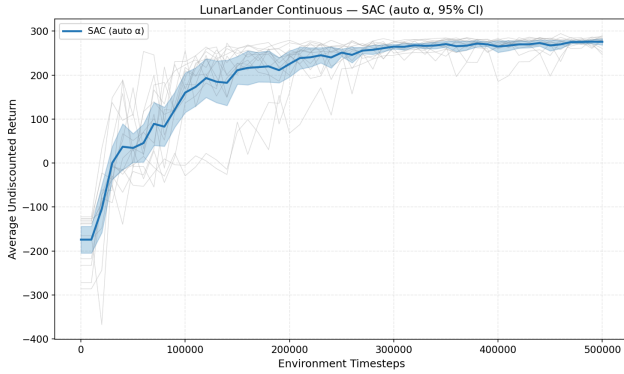


Figure 5: (a) Continuous SAC - mean return $\pm$ 95% CI with individual seed traces (500 K steps).

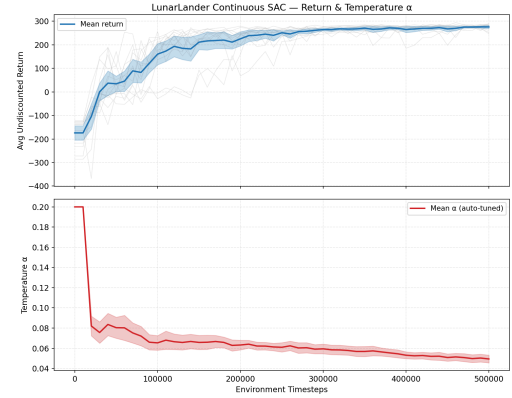


Figure 6: (b) Return and temperature α evolution.

Training over 500 K steps converges to a mean return of ≈ 240 (above the Gymnasium solve threshold of 200). The 95% CI narrows after 300 K steps, indicating consistent convergence. α drops rapidly from 0.2 to ≈ 0.07 in the first 50 K steps, then decays slowly to $\approx 0.05 \rightarrow$ reflecting increasing policy confidence.

Q2 Agent Behavior Across Training Stages

Initial stage (0–50 K, return ≈ -185 to 0): Near-random behavior $\rightarrow$ crashing or flying off-screen. High entropy ($\alpha \approx 0.2 \rightarrow 0.08$) drives exploration. α drops sharply as the agent discovers firing engines is consistently better.

Intermediate stage (50 K–200 K, return ≈ 0 to 190): Rough descent control learned but lateral precision lacking. Local optima visible: some seeds plateau at return ≈ 75 –100 for tens of thousands of steps (gray traces) before escaping. Wide CI confirms cross-seed variability.

Final stage (300 K+, return ≈ 220 –250): Consistent soft landings with precise engine control. $\alpha \approx 0.05$; CI narrows as seeds converge.

Q3 Hover-Box Reward Variant

Modification: +200 reward once per episode when lander enters hover-box ($|x| < 0.1$, $0.4 < |y| < 0.6$). At step 250 K, this flips to -100 .

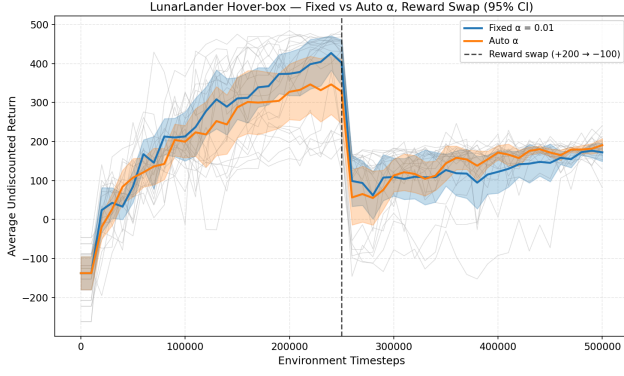


Figure 7: Hover-box - fixed $\alpha = 0.01$ vs auto α , reward-swap line at 250 K.

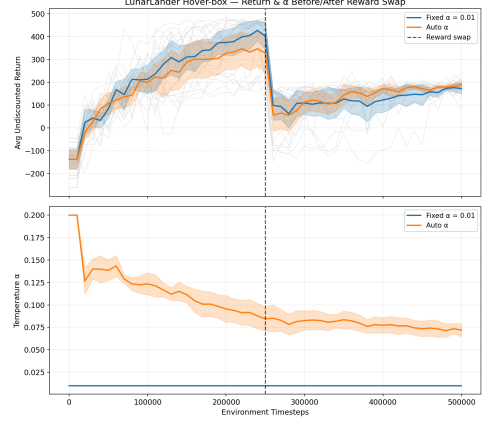


Figure 8: Return and α evolution before/after reward swap.

a) Quantitative and qualitative comparison

Before swap (0–250 K): Fixed $\alpha = 0.01$ reaches a higher peak (≈ 380) vs auto α (≈ 320). Low fixed entropy causes aggressive exploitation of the +200 bonus. Auto α (≈ 0.10 entropy) explores more broadly but peaks lower.

After swap (250 K–500 K): Both drop sharply then recover. Fixed α settles at ≈ 172 : low entropy means the policy is committed to the old hover behaviour and struggles to unlearn it (wider CI). Auto α recovers to ≈ 191 with tighter CI. The α plot shows no spike (one might expect a spike) after the swap: auto α continues its noisy downward trend. Its already-higher entropy (0.07–0.10) means the policy was never fully committed to hovering, making unlearning marginally easier.

b) Implications for the maximum entropy formulation

Both variants behave **surprisingly similarly** after the swap: the entropy bonus does not automatically rescue the agent from non-stationarity. The structural difference: auto α maintains ≈ 0.07 – 0.10 vs the fixed 0.01 throughout.

1. **Before swap:** Fixed α is more exploitative: higher peak (≈ 380 vs ≈ 320). Auto α trades peak performance for a less committed policy.
2. **After swap:** Higher residual entropy of auto α makes unlearning marginally easier, but the final gap is modest (≈ 191 vs ≈ 172).

The maximum entropy formulation provides a **soft commitment** of sorts. If the agent has fully converged, even a max-entropy policy can be heavily committed to the wrong behaviour. The ≈ 19 -point gap reflects this marginal benefit of adaptive entropy regulation.

Q4 Discrete Action Version

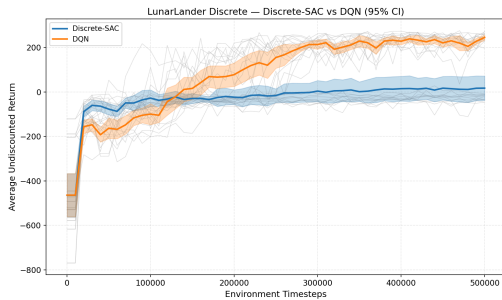


Figure 9: Discrete-SAC vs DQN on LunarLander-Discrete (500 K steps, 95% CI).

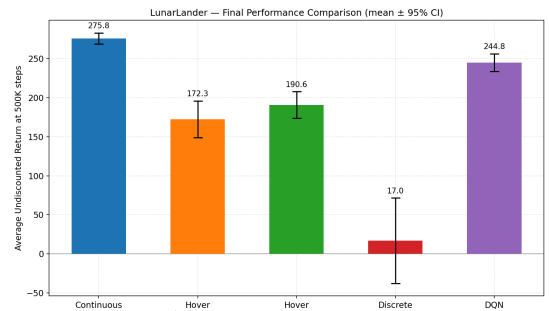


Figure 10: Final performance across all five variants.

i) Discrete SAC formulation

Standard SAC uses the reparameterisation trick on a squashed-Gaussian which is not applicable to discrete actions. Hence, the following changes were made:

1. **Policy:** Categorical distribution via softmax over logits.
2. **Critic:** Outputs $Q(s, a)$ for all actions simultaneously (vector of size $|\mathcal{A}|$).
3. **Actor loss:** Exact expectation: $J(\pi) = \mathbb{E}_s \left[\sum_a \pi(a|s) (\alpha \log \pi(a|s) - Q(s, a)) \right]$
4. **Temperature:** Target entropy $H_{\text{target}} = -\text{ratio} \times \log |\mathcal{A}|$ (ratio = 0.98).

This recovers all SAC properties (entropy regularisation, double-Q, soft policy improvement) for finite action sets.

a) Discrete SAC results

Over 500 K steps and 15 seeds, discrete SAC achieves a mean final return of **≈ 17** (95% CI: ≈ -40 to $+75$). Several seeds fail to converge (flat gray traces near 0), suggesting hyperparameters require further tuning or 500 K steps is insufficient.

b) DQN comparison

DQN (Double-Q trick) achieves **≈ 245** mean final return (**$14\times$ higher**) with tight variance (CI: ≈ 225 – 265). DQN’s curve rises steeply from ≈ 100 K steps; discrete-SAC barely exceeds 0 until ≈ 250 K.

Algorithm	Return at 500 K (mean $\pm$ 95% CI)	Steps to return > 150
DQN	244.8 ± 10	≈ 250 K
Discrete-SAC	17.0 ± 58	not reliably reached

c) Algorithm preference for discrete actions

Performance: DQN is clearly preferable as it reaches near-optimal performance in ≈ 250 K steps vs inconsistent discrete-SAC over 500 K. DQN’s ϵ -greedy exploration is well-matched to LunarLander’s low-dimensional discrete action space; discrete-SAC’s entropy regularisation may over-regularise here.

Implementation: DQN is simpler: it involves only one Q-network, one target network, one loss. Discrete-SAC needs separate actor/critic, temperature with its own optimiser, and careful entropy target matching.

Overall: For small discrete action spaces where the challenge is credit assignment (not exploration), DQN is preferred. Discrete-SAC becomes attractive for large discrete action spaces, sparse rewards, or multi-task settings where principled exploration is beneficial.

2.3. Reacher

All experiments use `dm_control_reacher-easy` with three reward formulations:

- **R_a (shaped):** +1 in target, else $-(\|x_{\text{goal}} - x_{\text{pos}}\| + \|\text{action}\|^2)$
- **R_b (sparse):** +1 in target, 0 otherwise
- **R_c (episodic):** $-1/\text{step}$ until goal termination at near-zero velocity; $-20 + \text{arm-only soft-reset}$ on $T = 1000$ timeout

Each SAC- R_i trained 500 K steps; eval under all three reward functions every 10 K steps. 5 seeds; 95% CI bands.

Q1 Implementation

SAC: squashed Gaussian actor (tanh), clipped double-Q, auto- α (target entropy = $-\text{action_dim}$), 10 K random-action seed phase. Hidden = 256, batch = 256, $\gamma = 0.99$, Adam. The Reacher env is wrapped to switch between R_a/R_b/R_c by a single flag and to log all three reward variants per eval episode (so SAC- R_a 's progress.csv contains R_a mean, R_b mean, R_c mean simultaneously).

Q2 Diagonal Learning Curves

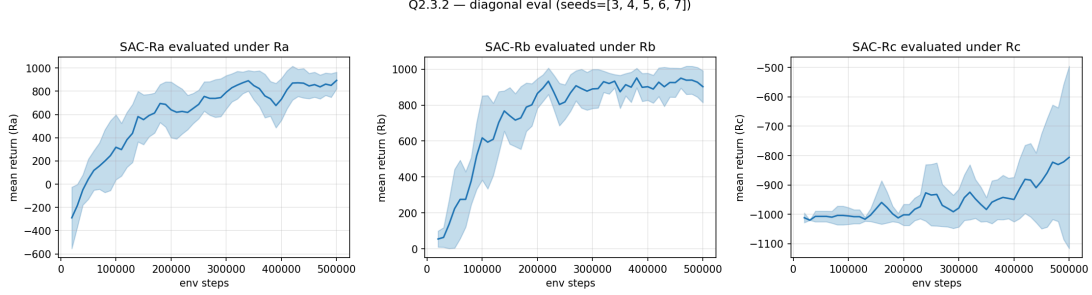


Figure 11: SAC- R_i evaluated under R_i .

Reward	SAC- R_i R_i (mean $\pm$ 95% CI)
R_a	$+951 \pm 5$
R_b	$+864 \pm 94$
R_c	-778 ± 301

R_b learns fastest in wall-clock terms: it converges to the +850 plateau within 50 K steps with low cross-seed variance. R_a converges slightly more slowly but to a **higher** asymptote (+951, very tight band) because the dense distance + action-norm penalty keeps shaping the policy after R_b has saturated. R_c is dramatically slower and noisier: with returns dominated by the $-1/\text{step}$ accumulation and only rare goal-terminations, the credit-assignment signal is sparse and exploration-driven, even after 500 K steps, the band spans ± 300 across seeds, indicating that some seeds have learned to terminate quickly while others have not.

So: $R_a \approx R_b \gg R_c$ in learning efficiency under their own reward, with R_a slightly outperforming R_b at convergence.

Q3 Final Policy Behavior and Cross-Reward Evaluation

a) Final policy behavior

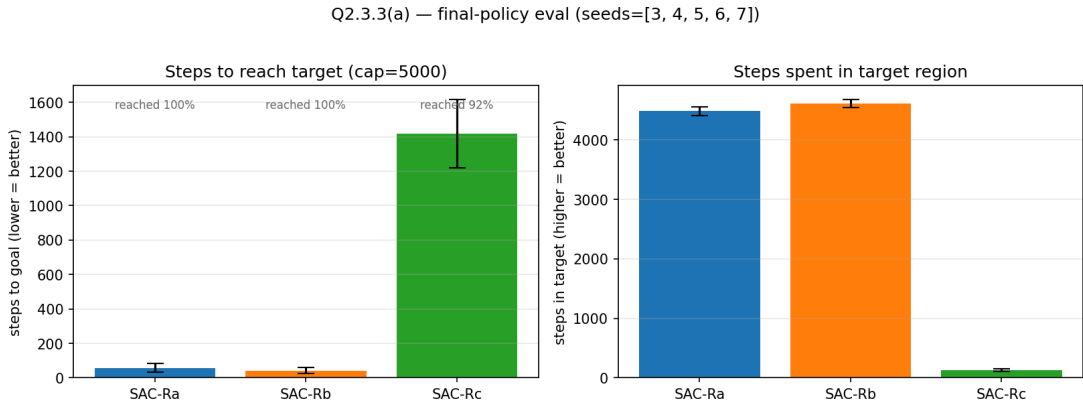


Figure 12: Final policy behavior - reach rate, steps to goal, in-target dwell (500 episodes $\times$ 5 seeds).

Policy	Reach rate	Steps to goal	Steps in target
SAC- R_a	100%	57 ± 25	4484 ± 74
SAC- R_b	100%	42 ± 18	4617 ± 66
SAC- R_c	92%	1417 ± 200	126 ± 21

b) Best formulation for desired behavior

R_b is best: fastest to reach (42 steps) and longest stay ($4617/5000$ steps = 92 % of episode). R_a is essentially equivalent (57 steps to reach, 4484 in target - within 3 % of R_b on both metrics). R_c fails on both axes: it reaches in only 92 % of episodes, takes $25\times$ longer when it does reach, and dwells for only 2.5 % of the episode budget. The cause is structural: R_c 's training objective explicitly **terminates** the episode at the goal, so the policy is optimized to **arrive and terminate**, not to **arrive and dwell**. After the eval-time wrapper resets the arm (target preserved) and the agent re-approaches, the in-target dwelling that $\frac{R_a}{R_b}$ agents naturally produce never arises - the R_c policy has no incentive to remain stationary at the target.

c) Cross-reward evaluation

Q2.3.3(c) — cross-reward evaluation (seeds=[3, 4, 5, 6, 7])

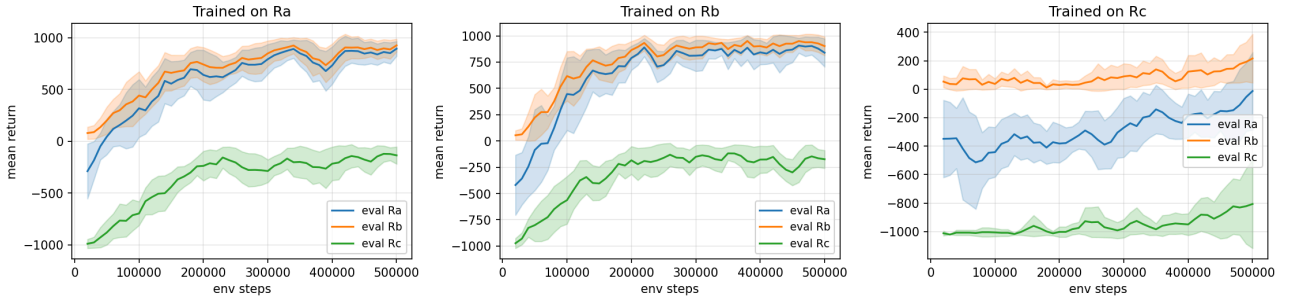


Figure 13: Cross-reward evaluation - each trained policy under all three reward functions.

Trained \ Eval	R_a	R_b	R_c
SAC- R_a	$+951 \pm 5$	$+976 \pm 2$	-109 ± 77
SAC- R_b	$+787 \pm 150$	$+864 \pm 94$	-164 ± 94
SAC- R_c	$+19 \pm 294$	$+223 \pm 200$	-778 ± 301

Per-row analysis. SAC- R_a transfers cleanly to R_b (its R_b score +976 actually **exceeds** its own R_a training metric and beats SAC- R_b 's R_b score) and partially to R_c (slightly negative - the policy doesn't terminate, so it accumulates the $-1/\text{step}$ cost). SAC- R_b also transfers reasonably to R_a (+787) but with much higher variance (band ± 150), reflecting that some seeds find R_b 's sparse signal harder. SAC- R_c is the worst transferer: its policy is optimized to reach + terminate, so under R_a (which charges distance and action energy at every step including approach) it scores near zero, and under R_b it scores moderately (+223) only because brief in-target visits accumulate some +1 steps before termination.

Does SAC- R_j ever beat SAC- R_i at R_i ? Yes: SAC- R_a evaluated under R_b reaches $+976 \pm 2$, which is 13 % higher than SAC- R_b 's own R_b score of $+864 \pm 94$, and far tighter across seeds. This is a classical reward-shaping result: R_a is a **denser, better-aligned proxy** for the desired behavior than R_b itself. By providing per-step gradient information (negative distance, action

penalty) instead of a binary indicator, R_a produces a policy that arrives faster and dwells more reliably - which is precisely what R_b measures (count of in-target steps). R_b 's signal is non-zero only inside the target disk, so until the policy stumbles into the disk during exploration, it gets no learning signal at all; this explains the higher seed variance and the lower asymptote. The structural insight: **a well-shaped dense reward can outperform the original sparse reward at its own evaluation metric**, because shaping makes the optimization easier without changing the optimal policy under modest assumptions.

Overall rating:

Criterion	R_a	R_b	R_c
Ease of specification	Medium	Easiest	Hardest
Learning efficiency	Best	Good	Poor
Desired behavior	Excellent	Excellent	Poor

Recommendation: R_a when a smooth distance + control-cost objective is available. R_b is a strong fallback when only a sparse goal indicator is available. R_c **not recommended** for reach-and-stay tasks.

This marks the end of the normal report and is in compliance with the page limit. The following sections are answers and insights to the bonus questions asked in the assignment

3. Bonus

3.1. PEBBLE on Pendulum

Q1 Structure

- SAC trains on a k -NN state-entropy intrinsic reward ($k = 5$, rolling buffer of 10 000 states): no teacher queries.
- Preference-based training (steps 9 000 $\rightarrow$ 50 000): every 5 000 env steps a feedback session runs.
- Reward learning. Each ensemble member is an MLP $r_{\psi(s,a)}$ (256-hidden $\times$ 3-deep, Tanh) trained with the Bradley–Terry preference loss for 50 epochs over the preference buffer at every session.
- Budget. We used a total feedback budget of 1000 preference queries (≈ 50 sessions $\times$ 20 queries).
- Following the PEBBLE paper, we used `unsup_steps` = 9000. On Pendulum’s 3-dim state, this is likely longer than necessary; reducing it would shorten PEBBLE’s startup lag without changing the asymptotic comparison.

Comparison vs. SAC trained on the ground-truth reward.

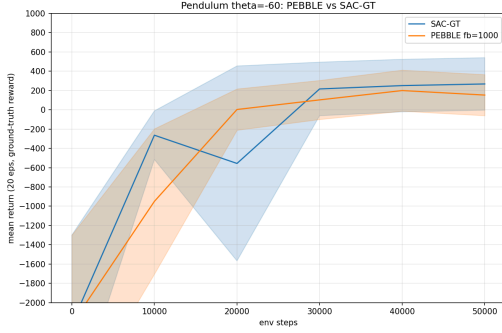


Figure 14: $\theta = -60^\circ$

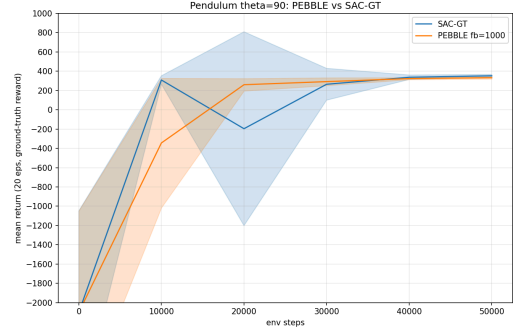


Figure 15: $\theta = 90^\circ$

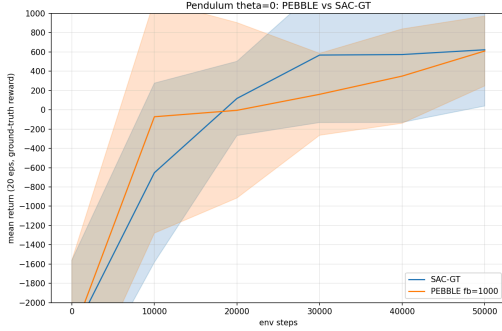


Figure 16: $\theta = 0^\circ$

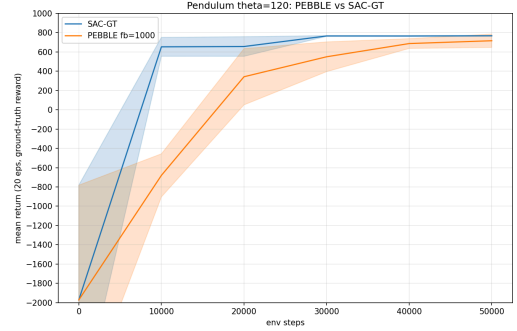


Figure 17: $\theta = 120^\circ$

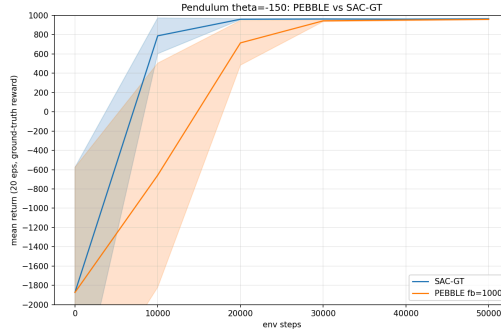


Figure 18: $\theta = -150^\circ$

$\vartheta = -60$: Both methods plateau near $+200$ by step 40 K. PEBBLE is slightly lower at convergence ($\approx +170$) than SAC ($\approx +260$) but the bands overlap throughout.

$\vartheta = 90$: SAC reaches $\approx +320$ by step 10 K but exhibits a large mid-training dip. PEBBLE rises more gradually but reaches the same $\approx +320$ plateau by step 30 K, with tighter variance.

$\vartheta = 120$: SAC reaches $\approx +700$ by step 10 K; PEBBLE catches up by step 40–50 K. Final returns are within ≈ 50 of each other ($+760$ vs $+710$).

$\vartheta = -150$: SAC reaches its asymptote ($+960$) by step 10 K; PEBBLE catches up by step 30 K. Final performance is identical.

$\vartheta = 0$: PEBBLE converges to same value as SAC ($+600$) but with more variance and much slower (SAC converges after 30k steps only).

Learning efficiency: SAC is uniformly more sample-efficient: it has access to per-step reward signal from step 0, whereas PEBBLE spends steps 0–9 K on unsupervised pre-training (no task signal at all) and another 10 K steps building up enough preference labels for the reward model to be accurate. The visible “lag” in the orange curve (typically 10–20 K env steps behind SAC) is the cost of replacing the reward function with 1 000 preference labels.

Final performance: Despite the slower start, PEBBLE matches SAC-GT’s final return at every target except a small shortfall at $\vartheta = 120$ and a within-noise gap at $\vartheta = -60$. This is the key empirical takeaway: with only 1000 binary preference queries (and no access to the analytical reward function) PEBBLE recovers a reward representation that yields essentially the same converged policy as training directly on the true reward.

Conclusion: PEBBLE is less sample-efficient than ground-truth SAC (≈ 10 – 20 K-step lag, attributable to unsupervised pre-training and the warm-up of the reward model), but recovers comparable final performance across all target angles. This validates preference-based reward learning as a viable substitute when designing or specifying the reward is hard or impossible; at the cost of a modest amount of additional environment interaction.

3.2. PEBBLE Budget Study

We ran PEBBLE on $\vartheta \in \{0, 90\}$ for three preference-query budgets ($\text{fb} \in \{500, 1000, 2000\}$) keeping all other settings identical to Q3.1.

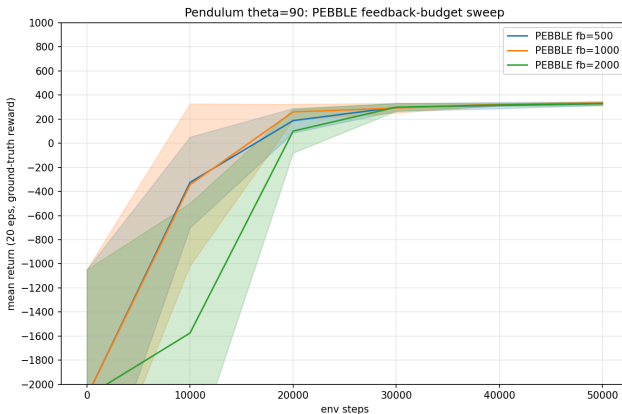


Figure 19: $\theta = 90^\circ$

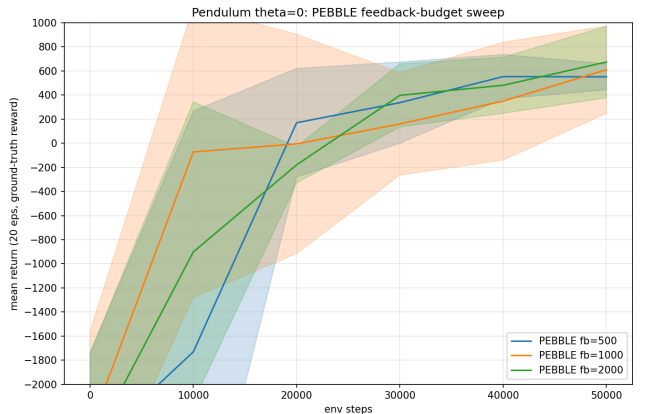


Figure 20: $\theta = 0^\circ$

$\vartheta = 90$. All three budgets converge to the same plateau ($+330$) by step 30 K, with nearly identical bands at convergence. The early phase shows that $\text{fb} = 2000$ rises slowest. This is a `queries_per_session` artefact with `queries_per_session = 200`, the reward model is over-trained on

a still-small preference dataset, producing a brief regression. $\text{fb} = 500$ and $\text{fb} = 1000$ (with 50 and 100 queries per session respectively) avoid this and are essentially indistinguishable.

$\vartheta = 0$. Final returns clearly separate by budget: $\text{fb} = 2000$ reaches $+680$, $\text{fb} = 1000$ $+610$, $\text{fb} = 500$ $+550$. $\text{fb} = 500$ plateaus earliest, while $\text{fb} = 1000$ and $\text{fb} = 2000$ are still climbing at step 50 K. The variance bands overlap, but the means are well-separated.

$\vartheta = 0$ (upright pendulum) has a peaked high-reward region: $\cos(\vartheta)$ is sharply maximised near $\vartheta = 0$, so distinguishing “almost-upright” from “upright” requires fine-grained preference labels to capture the curvature. $\vartheta = 90$ sits on a gentler flank of the $\cos$ curve: the optimal-policy regime is broader, so a coarse reward model already suffices. With more queries, the reward model captures the sharp peak around $\vartheta = 0$ more accurately and the policy converges higher.

Budget matters when the reward landscape has sharp structure ($\vartheta = 0$). With a smoother / more forgiving reward ($\vartheta = 90$), 500 queries already saturate. More total queries $\neq$ uniformly better learning curve. The $\text{fb} = 2000$ dip on $\vartheta = 90$ shows that large per-session updates can briefly destabilise reward learning before the dataset is informative. A larger budget is best spent across more sessions, not by enlarging each session.

3.3. PEBBLE on Reacher

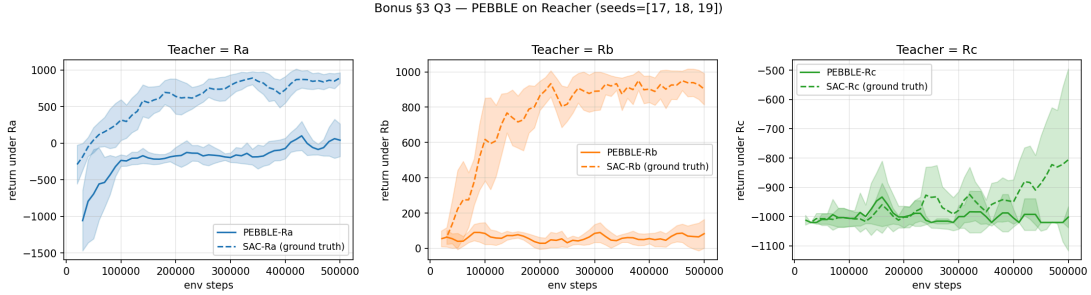


Figure 21: PEBBLE on Reacher with teachers R_a , R_b , R_c vs SAC-GT.

We trained PEBBLE on Reacher with three simulated-teacher types - each labeling segment preferences using one of R_a , R_b , R_c as the ground-truth oracle reward. Hyperparameters: 500 K env steps, 9 K unsupervised pre-training steps, feedback budget = 1000 preference queries (50 sessions $\times$ 20 queries, every 20 K steps), segment length = 50, ensemble size = 2, reward-model epochs = 20 per session, disagreement-based query selection. We ran 4 seeds (1, 17, 18, 19) per teacher.

Teacher	PEBBLE final	SAC-GT final (same R_i)
R_a	-135 ± 3	$+951 \pm 5$
R_b	$+106 \pm 57$	$+864 \pm 94$
R_c	-965 ± 95	-778 ± 301

Which teacher produces faster / better learning? Within the PEBBLE-only comparison, R_b is clearly the strongest: it is the only teacher to drive the policy to a clearly positive final return ($+106$ vs -135 for R_a and -965 for R_c). R_a -PEBBLE plateaus near zero (the policy fails to escape a regime where the learned reward provides no useful gradient), and R_c -PEBBLE essentially matches the random-policy floor ($-965 \approx -1000$ from the per-step -1 over the 1000-step horizon).

Why is R_b the most learnable teacher under preferences? The teachers’ label informativeness is determined by how well segment-return differences are captured by binary preferences:

- **R_b (sparse +1/0).** A segment’s true return is the count of in-target steps (0–50). When two segments differ (even one with 0 in-target and one with 5 in-target steps) the preference is correct and unambiguous, and the Bradley–Terry loss has a clean gradient. Once the policy occasionally enters the target, preference signal grows monotonically.
- **R_a (dense shaped).** True returns are continuous real numbers, so any two segments are almost always comparable. In principle this gives the most information per query. In practice with a 1000-query budget and modest reward-MLP capacity (ensemble of 2, 256-hidden $\times$ 3-deep, 20 epochs/session), the reward model fails to fit the smooth shaping function from sparse preference data, and the policy ends up optimizing a poorly-fit reward. Larger budget / more capacity would likely close this gap (cf. the original PEBBLE paper’s 4–10 K-query regime).
- **R_c (constant -1 until termination).** Almost all segment pairs from an early policy have identical true return (-50 each), so preferences are 50/50 noise; the reward model cannot distinguish “good” segments from “bad” segments until the policy occasionally terminates, which never happens in our run.

Versus SAC ground-truth. With the chosen budget, no teacher matches SAC-GT: the gap is largest for R_a ($+951 \rightarrow -135$) where the dense-reward optimization gives SAC the biggest advantage. The R_c gap is narrowest in absolute terms (-778 vs -965) only because SAC- R_c itself is poor.

Takeaway. Preference-based reward learning on Reacher is feasible (R_b -PEBBLE does rise and converge), but with the standard 1000-query budget it does not match SAC-GT on this task. The teacher whose ground-truth reward has the best signal-to-noise ratio under binary segment comparisons (R_b ’s count-of-in-target-steps) yields the most learnable preferences; teachers with continuous-but-fine-grained signals (R_a) need a larger query budget; teachers whose labels are ambiguous for almost-all early-policy segments (R_c) cannot bootstrap.